

轮对话都在背后生效。User prompt 是用户每次输入的内容。作为算法工程师，两种你都要会写。

3. 写 prompt 最核心的原则是什么？

把你脑子里已经知道、但没有写出来的信息，显式地告诉模型。你觉得不用说的背景信息，比如写给谁、什么场景、什么格式、什么不能做，模型全都不知道，除非你写出来。

4. 好 prompt 是一次写出来的还是改出来的？

改出来的。跑一遍、看输出、找差距、改 prompt、再跑。每次改完要用一组覆盖不同场景的测试用例全跑一遍，防止“改好了这个场景，改坏了那个场景”。

5. 什么时候 prompt 够用，什么时候该考虑微调？

当任务逻辑你能用自然语言说清楚、不依赖特别冷门的专业知识、对一致性要求不极端时，prompt 通常就够了。当你发现瓶颈出在模型知识的缺失、格式遵循的不稳定、或者规则复杂到 prompt 装不下时，再考虑微调。但在此之前，先确认 prompt 已经优化到了极限。

13.2.11 Python 交互实验室

这一节讲的所有技巧——消息结构、五要素组装、few-shot 示例、模板替换——在你从网页聊天走向 API 开发的那一刻，都会从文字概念变成需要写成代码的东西。资源包中的 `prompt_demo.py` 用五个演示帮你提前走完这段路：你会看到一个精心组装的 system prompt 在代码里长什么样、few-shot 示例是怎么以 user-assistant 交替的方式塞进消息列表的、多轮对话历史是怎么一轮一轮挤占上下文窗口的。运行之后，建议你试两件事：把 few-shot 示例从三个改成一个，对比 token 开销的变化；然后把上下文窗口参数从 8192 改成 4096，看看多轮对话在第几轮就会撞到上限。这两个数字会帮你建立起“prompt 长度就是真金白银”的成本直觉。

13.3 结构化输出：怎么让大模型稳定地输出 JSON

上一节讲的是怎么把任务、背景和约束写进 prompt，让模型更容易答到点上。但很多工程场景里，答得像那么回事还不够，输出还得能被程序稳定解析。自然语言给人看，结构化输出给系统用，这一节就专门处理这个问题。

13.3.1 你以后天天要用的东西

这一节的内容，可能看起来不像 Transformer、不像注意力机制那样高大上。它不涉及什么精妙的数学推导，也不会出现在顶会论文的标题里。但我可以非常负责任地告诉你：在你真正坐到工位上干活的头三个月里，这一节讲的东西，你用到的频率可能比前面学的所有理论知识加起来还高。

为什么这么说？因为在真实的项目里，大模型的输出几乎从来不是给人直接看的。它的输出是要接到一条流水线里——被程序解析、被数据库存储、被下游系统消费。而程序不会理解自然语言。程序需要的是严格的、可预期的、结构化的数据格式。JSON 是目前最通用的那种格式。

你会在各种各样的场景里碰到这个需求。做智能客服的时候，用户发来一条消息，模型要从里面提取意图和关键信息，输出一个 JSON，传给后台系统自动处理。用大模

型批量生成训练数据的时候，每一条数据都必须是严格的 JSONL 格式，差一个逗号都喂不进训练脚本。用大模型做数据清洗和标注的时候，模型输出的分类标签、置信度、标注理由必须是结构化的，才能批量写进数据库供后续分析。后面如果接触到 Agent（智能体）的开发，模型需要判断“该调用哪个工具、传什么参数”，这个判断结果也是一个 JSON。

而且不仅仅是 JSON。XML、YAML、CSV、自定义的模板格式，凡是结构化的输出，面临的问题和解决思路都是万变不离其宗的。这一节用 JSON 做主角，但你学到的方法论可以平移到任何结构化格式上。

我之所以纠结了很久最终决定把这一节加上，就是因为这个话题太实战了。它不会出现在任何教科书的目录里，但你第一天上班就会碰到，而且你碰到之后会发现，网上搜到的零碎经验帖根本帮不了你建立系统性的认知。我希望你读完这一节之后，面对结构化输出的问题，脑子里有一张清晰的地图，知道有哪些方案、每个方案保证了什么、没保证什么、什么时候该用哪个。

13.3.2 先看一个场景

想象你在一家做智能客服的公司，产品上线一周了，老板让你加一个新功能：用户发来一条退货请求，模型自动从里面提取关键信息，也就是商品名称、订单号、退货原因，然后输出一个 JSON，传给后台系统生成退货工单。

你觉得这不难。模型理解能力挺强的，提取几个字段嘛，写个好点的 prompt 就行了。你花了半小时调好 prompt，模型的回答看起来很完美：

```
商品名称：蓝色运动外套
订单号：20250415123
退货原因：尺码偏大
```

你把这个结果拿给后端同事看，他问你一个问题：“这个输出我怎么解析？”

当然，真要硬来，也可以按冒号和换行写规则去拆。但这种规则很脆：字段顺序一变、模型多说一句话、用户原因里自己带了换行，解析就可能出错。工程里更稳的做法，是让模型输出一种大家都认的固定格式，比如 JSON。它长这样：

```
{"product": " 蓝色运动外套", "order_id": "20250415123", "reason": " 尺码偏大"}
```

程序拿到这个 JSON 之后，能精确地知道 `product` 对应什么、`order_id` 对应什么，然后把它们分别填进数据库的不同字段里。整个过程是自动的，中间没有人。

但如果模型的输出不是这个格式，哪怕只是少了一个引号、多了一个逗号、或者在 JSON 前面多说了一句废话，后端的 Python 代码在做 `json.loads()` 的时候就会直接报错。报了错，退货工单就生成不了。用户等半天没回音，投诉就来了。

好，你意识到了问题的严肃性。那就改 prompt，明确要求模型输出 JSON 格式。这是所有人的第一反应，也确实是实战中的起点。

13.3.3 在 prompt 里说清楚

你在 prompt 里写得很详细：告诉模型“你只能输出一个 JSON 对象，不要输出任何其他文字”，列出每个字段的名称和类型，还给了一个完整的 JSON 示例。一个典型的 prompt 可能长这样：

你是一个信息提取助手。用户会发来一段退货请求，你需要从中提取关键信息，以 JSON 格式输出。

输出格式要求：

- 只输出一个 JSON 对象，不要输出任何其他文字
- JSON 必须包含以下字段：
 - product: 商品名称（字符串）
 - order_id: 订单号（字符串）
 - reason: 退货原因（字符串）

输出示例：

```
{"product": " 黑色羽绒服", "order_id": "20250101456", "reason": " 拉链损坏"}
```

你拿这个 prompt 跑了几条测试数据，模型表现得很乖，每次都老实地输出一个漂亮的 JSON。你觉得问题解决了。

然后你自信满满地跑了 500 条真实的用户退货消息。翻车了。

13.3.4 翻车的六种姿势

打开日志一条一条看，发现翻车的姿势五花八门。

翻车一：模型忍不住说废话

这是出现频率最高的一种。你明明说了“不要输出任何其他文字”，但模型有时候就是忍不住：

好的，以下是提取结果：

```
{"product": " 蓝色运动外套", "order_id": "20250415123", "reason": " 尺码偏大"}
```

或者在 JSON 后面加一句：

```
{"product": " 蓝色运动外套", "order_id": "20250415123", "reason": " 尺码偏大"}  
希望对你有帮助！如果还有其他问题请随时告诉我。
```

不管是前面多了一句话还是后面多了一句话，你的 Python 代码拿整段文本去做 `json.loads()`，都会直接报错。JSON 解析器期望的输入是一个纯粹的 JSON 字符串，前面那句“好的，以下是提取结果：”在它眼里是非法字符。

为什么模型会这样？因为它在指令微调阶段被大量教导“要礼貌、要有帮助、要解释自己在做什么”。这些行为在对话场景里是优点，但在结构化输出场景里就成了捣乱：模型的礼貌本能和你的“只输出 JSON”指令在互相拉扯，偶尔礼貌会赢。

还有一种更隐蔽的废话：模型用 Markdown 的代码栅栏把 JSON 包了起来：

```
```json  
{
 "product": " 蓝色运动外套",
 "order_id": "20250415123",
 "reason": " 尺码偏大"
}
```
```

肉眼看上去挺规整的，但那三个反引号加上 `json` 这个标记，对 JSON 解析器来说都是非法字符。如果你没有针对这种情况做预处理，一样会报错。那简单啊，解析前先把代码栅栏剥掉不就行了？确实，很多项目最后都会写这类预处理。但这种翻车真正阴

险的地方不在这里：模型版本更新之后可能突然开始偏好用代码栅栏包裹输出，你在旧版本上测的好好的 prompt，换了新版本突然一大片报错，你去查日志才发现罪魁祸首是三个反引号。

实战避坑 · 锁定模型版本，否则你的线上系统会被“静默升级”搞崩

你用某家 API 跑通了结构化输出，上了线，稳定运行了一个月。某天早上你打开监控，发现格式报错率突然从 0.5% 飙到了 8%。你的代码没改过，prompt 没动过，数据也没变。查了两个小时才发现：API 提供商在后台更新了模型版本，新版本的输出风格变了，比如开始偏好用 Markdown 代码栅栏包裹 JSON。在调用 API 时，永远指定具体的模型版本号（比如用 `gpt-4o-2024-08-06` 而不是 `gpt-4o`），不要用会自动升级的别名。模型更新是好事，但应该由你主动测试后再切换，而不是被动地被“惊喜”。

翻车二：字段名乱跑

你要求的字段叫 `product`，但模型不一定每次都用这个名字。有时候它会这样：

第一条输出：`{"product": "蓝色运动外套", ...}`

第二条输出：`{"product_name": "红色连衣裙", ...}`

第三条输出：`{"商品名称": "黑色皮鞋", ...}`

三种写法，每种单独看都说得通。但你的下游程序只认 `product` 这个名字，代码里写的是 `data["product"]`，模型输出 `product_name` 的时候，程序找不到 `product` 这个 key，直接抛 `KeyError`。

这种问题在输入内容比较复杂的时候尤其容易出现。模型看到用户的原文里写了“商品名称”四个字，就好心地把字段名也改成了中文，完全没理会你在 prompt 里定义的英文字段名。

翻车三：值的类型漂移

你期望价格字段是一个数字 `99.5`，模型给你输出了一个字符串 `"99.5元"`，还带单位。或者你期望一个布尔值 `true`，模型输出了 `"是"`。

期望的：`{"product": "蓝色外套", "price": 99.5, "in_stock": true}`

实际的：`{"product": "蓝色外套", "price": "99.5 元", "in_stock": "是"}`

你的下游代码拿到 `price` 之后要做数值计算，`"99.5元"` 这个字符串参与加减乘除会直接报类型错误。如果某个字段只能从几个固定值里选，比如 `positive`、`negative`、`neutral`，解析之后还要检查它是不是这三个值之一，不在名单里的就算格式对了也不能放行。

翻车四：JSON 语法错误

这一类翻车肉眼很难看出来。你盯着模型输出看了半天觉得没问题啊，但解析器就是报错。典型的：

末尾多了一个逗号：

```
{"product": " 蓝色外套", "order_id": "20250415123", "reason": " 尺码偏大",}
```

最后一个字段后面那个逗号，在 JavaScript 里是合法的（叫 trailing comma），但在 JSON 标准里是非法的。如果你写过 JavaScript，你可能觉得这完全没问题，但 JSON 解析器不这么想。

用了单引号：

```
{'product': ' 蓝色外套', 'order_id': '20250415123'}
```

Python 的字典用单引号没问题，但 JSON 标准要求必须用双引号。模型在训练数据里见过大量 Python 代码，偶尔会把 Python 字典的写法带到 JSON 输出里来。

还有一种更折磨人的情况，是 JSON 里面套 JSON。比如你在做数据生成，让模型造一条 SFT 训练样本，外层格式是 `{"instruction": "...", "output": "..."}` ，而 `output` 的值恰好又是一段 JSON 字符串。这个时候，内层 JSON 的双引号必须转义成 `\`，否则外层解析器会在内层双引号那里提前断句。模型很容易在这里翻车：有时漏转义，有时多转义，有时把 `\` 写成 `\\`。遇到这种需求，别硬让模型一次性写对所有转义。更稳的做法是分两步：先让模型输出内层内容，再由你的代码把它塞进外层 JSON，并负责转义。转义这种机械活，交给代码比交给模型可靠得多。

翻车五：截断

这种特别坑，因为报错信息不直观。模型输出的内容太长，撞上了你设置的 `max_tokens` 上限，被从中间硬生生切断了：

```
{"product": " 蓝色运动外套", "order_id": "20250415123", "reason": " 尺码偏
```

后面的引号、花括号全没了。你拿到的是一个残缺的 JSON 片段，根本无法解析。

翻车六：幻觉字段

你只要求三个字段，模型自作主张加了第四个：

```
{"product": " 蓝色运动外套", "order_id": "20250415123", "reason": " 尺码偏大",  
"confidence": 0.95}
```

`confidence` 是哪来的？你没要求这个字段，模型自己觉得加一个置信度挺合理的，就给你加上了。

另一种失败方向也要单独记住：模型可能觉得某个字段在这条数据里“不适用”，干脆跳过不输出。你的代码去取这个字段的时候，同样会遇到 `KeyError`：前一种是多了不该有的字段，后一种是缺了应该有的字段。

你可能会问：这些翻车是因为模型太笨吗？

不是。哪怕你用目前最强的模型，把 prompt 打磨到极致，只要跑的量够大，这些问题都会出现。原因在于模型生成文本的根本工作方式：它在生成每一个 token 的时候，你的 prompt 只是它参考的众多上下文信号之一。prompt 说“只输出 JSON”，这个信号确实能把模型推向正确的方向，但它不能保证每一步、每一次都占据绝对主导。一旦某一步生成偏了，比如多输出了一个“好”字，后续的生成就会顺着这个偏差继续走。

模型不会在生成过程中突然“醒过来”说“哎不对我刚才不该说那个字”。错误会被放大，而不是被纠正。

本质上，prompt 约束是一种软约束：它提高了正确格式出现的概率，但无法提供确定性的保证。

那怎么办？再写得更长一点、更凶一点，prompt 里强调三遍“只输出 JSON”？你已经试过了，真的不行。更稳的方向，是尽量把格式约束前移到生成过程里：不是在模型生成完之后去检查、去修补，而是在它生成每一步的时候，就不让它走错路。

13.3.5 从源头堵死：受限解码

前面所有的方案都是同一个思路：让模型自由发挥，出了问题再去擦屁股。现在要讲的方案彻底换了一个方向：在模型生成的过程中，从每一步就限制它只能走合法的路。

回忆一下前面讲解码策略时说过的流程：模型在每一步，会为词表里的所有 token 打分（这些分数就是 logits），然后经过 Softmax 变成概率分布，最后从这个概率分布中采样出一个 token。

受限解码插手的位置，就在 Softmax 之前。这里一定要分清楚：模型只负责给所有候选 token 打分；判断哪些 token 合法的，是外面的解码器、API 或推理框架。它手里拿着一套 JSON 语法规则，会根据已经生成的前缀判断当前处在什么状态：是在等 key，等冒号，等 value，还是正在字符串中间。状态一确定，下一步哪些 token 合法也就确定了。

所以，受限解码做的事情是：在 Softmax 之前，把那些一选就会破坏 JSON 语法的 token 的分数强制设成负无穷大。

负无穷经过 Softmax 之后变成 0。那些不合法的 token 被彻底从候选名单上移除了，模型不可能选到它们。

假设模型已经生成了 {"age": 这几个 token，现在要决定下一个 token 是什么。简化一下，假设词表里只有 5 个候选 token：

| token | 在当前位置是否合法 | 模型给的原始分数 |
|-------|-------------|----------|
| 2 | 合法（数字值的开头） | 3.0 |
| " | 合法（字符串值的开头） | 2.5 |
| hello | 不合法 | 4.0 |
| } | 不合法 | 1.0 |
| , | 不合法 | 0.5 |

注意一个关键细节：hello 的原始分数最高，是 4.0。模型最“想”输出的就是它。但 {"age": hello 不是合法的 JSON。JSON 的值要么是数字，要么是用双引号包裹的字符串，要么是 true/false/null，裸单词 hello 不在合法选项里。} 不合法的原因不同：age 的值还没给，对象不能就这么结束。，也一样，值还没给，不能跳到下一个字段。

如果不做任何干预，直接对这 5 个分数做 Softmax：

$$e^{3.0} \approx 20.1, e^{2.5} \approx 12.2, e^{4.0} \approx 54.6, e^{1.0} \approx 2.7, e^{0.5} \approx 1.6$$

总和约 91.2。

算出的概率分布：2 占 22%，" 占 13.4%，hello 占 59.9%，} 占 3%，, 占 1.8%。hello 有将近 60% 的概率被选中。一旦选了它，JSON 就坏了。

加上受限解码之后，先把不合法的 token 的分数设为 $-\infty$ ：

修改后的分数： $[3.0, 2.5, -\infty, -\infty, -\infty]$

$e^{-\infty} = 0$, 所以 Softmax 的计算只剩两个有效的数字： $e^{3.0} + e^{2.5} \approx 20.1 + 12.2 = 32.3$ 。

新的概率分布：2 占 62.2%，" 占 37.8%，其余三个都是 0%。

模型现在只能在输出一个数字和开始一个字符串之间选择。不管选哪个，JSON 的语法都不会被破坏。

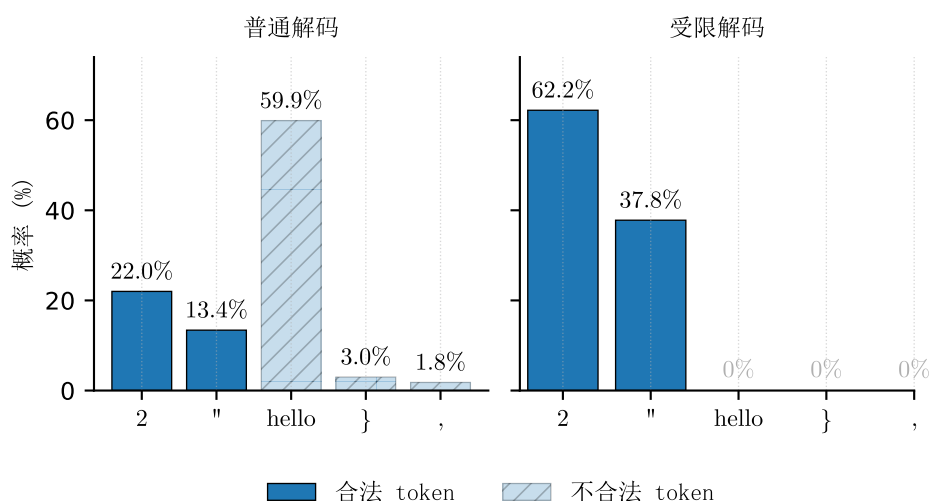


图 13.4: 受限解码的效果对比。模型已生成 `{"age":`，正在决定下一个 token。左图为普通解码：不合法的 `hello` 以 59.9% 的概率占据主导（斜线标记），一旦被选中，JSON 语法即被破坏。右图为受限解码：所有不合法 token 的概率被强制设为零，模型只能从合法的 `2`（数字值）和 `"`（字符串开头）中选择。

图 13.4 直观地展示了这个变化。左右两张图的对比，就是受限解码做的全部事情。

这就是受限解码做的全部事情：把不合法的选项从菜单上划掉。模型原本想输出什么没有变（原始 logit 不变），只是犯错的自由被拿走了。就像把一道填空题变成了选择题：填空题里你什么都能填，填错了就是语法错误；选择题里每个选项都是合法的，你随便选都不会出格。

在计算机科学里，这个负责记住当前语法状态的东西叫做“状态机”。JSON 的语法规则是固定的，所以这个状态机可以提前写好，在模型生成每一步时查一下就行，额外的计算开销很小。

但有一个至关重要的点必须说清楚：受限解码只保证语法正确，不保证内容正确。

在刚才的例子中，模型被限制在 `2` 和 `"` 之间选了 `"`，然后继续生成了 `"二十五岁"`。最终输出是 `{"age": "二十五岁"}`，语法完全合法，`json.loads()` 解析毫无问题。但如果你的下游系统期望 `age` 是一个整数 `25` 而不是字符串 `二十五岁`，这个值就是错的。受限解码管得了括号有没有闭合，管不了模型填的内容对不对。

这个区分非常重要，后面还会专门讲。先继续往下走。

面试考点 · 受限解码会不会降低生成质量

常见问法：“你说受限解码把不合法 token 的概率设为零，那这样不就干预了模型本来的概率分布吗？会不会导致模型输出的内容变差？”

典型错误：要么一口咬定“不会有任何影响”，要么夸大影响说会严重降低质量。

正确思路：影响确实存在，但在结构化提取场景下通常可以忽略。关键要分两种情况。第一种，模型正在输出 JSON 的“骨架”部分（花括号、冒号、引号、字段名），这些位置合法的 token 本来就很少，受限解码基本不会改变模型的选择。第二种，模型正在输出字段的“值”部分（比如在 `"name": "` 后面填内容），这时候大部分词表 token 都是合法的（因为它们都可以作为字符串的一部分），受限解码几乎不做任何干预。真正可能出问题的是极端情况：Schema 非常复杂，模型被迫生成一个它“不想”生成的嵌套结构，这时它在结构骨架上被迫选了低概率 token，可能影响后续内容的连贯性。但对于典型的信息提取任务（字段不多、嵌套不深），这个影响微乎其微。

13.3.6 API 帮你做好了

受限解码是底层原理。你可能在想：我需要自己写一个 JSON 状态机吗？

好消息是，如果你用的是主流 API，不需要。API 提供商已经把这件事封装好了，暴露成简单的参数让你一键开启。目前主流的封装分成两个层级，很多人容易搞混，这里要仔细区分。

第一个层级：JSON Mode

你在调用 API 的时候，加一个参数告诉它“我要 JSON 格式的输出”。API 保证返回的一定是语法合法的 JSON，不会多废话、不会少括号、不会有尾逗号。

但注意：它只保证“是合法的 JSON”，不保证“是你想要的那种 JSON”。

什么意思？你的 prompt 要求提取商品名称、订单号和退货原因，你开了 JSON Mode，模型返回了这个：

```
{"result": " 用户想退一件蓝色运动外套，订单号 20250415123，原因是尺码偏大"}
```

这是合法的 JSON 吗？完全合法。一个对象，一个字段 `result`，值是一个字符串。`json.loads()` 解析毫无问题。

但这是你想要的结构吗？完全不是。你要的是 `product`、`order_id`、`reason` 三个独立字段，模型把所有信息糊成了一句话塞进了一个字段里。你的下游程序去取 `data["product"]` 的时候，`KeyError`，字段不存在。

很多人第一次用 JSON Mode 的时候就在这里栽跟头：“不是说保证 JSON 了吗？为什么我的程序还是报错？”程序报的是“找不到 `product` 这个字段”，并不是 JSON 解析错误。JSON Mode 保证了语法，没保证结构。

第二个层级：JSON Schema 模式

这个层级比 JSON Mode 更进一步。你不仅告诉 API “我要 JSON”，还传入一份完整的 JSON Schema。

JSON Schema 是什么？它本身也是一段 JSON，只不过它描述的是“数据应该长什么样”的规则：哪些字段必须有、每个字段的名称叫什么、值是什么类型（字符串、数字、布尔值还是数组）、嵌套结构是什么样的。你可以把它理解成一份验收标准：模型的输出必须完全符合这份标准，否则 API 不会放行。

API 拿到你传入的 Schema 之后，会在底层构建一个状态机，和前面讲的受限解码思路一致。在模型生成的每一步，它只让模型选那些既符合 JSON 语法、又符合你 Schema 定义的 token。只要你在 Schema 里把必填字段、类型和额外字段规则写清楚，模型就不能少字段、不能乱加字段、不能搞错类型。

这才是真正意义上的想要什么格式就得到什么格式。

两个层级的区别用一句话概括：**JSON Mode** 保证“这是一个合法的 JSON”，**JSON Schema** 模式保证“这是一个符合你定义的那种合法 JSON”。前者管语法，后者管语法加结构。

不同 API 提供商对这两个层级的命名和参数写法各不相同，变化也快，查文档就能找到，不值得死记。重要的是理解这两个层级各自保证了什么、没保证什么，这个认知不会过时。

13.3.7 Function Calling: 模型不只是输出 JSON，还帮你决定该做什么

到目前为止，所有方案的目标都是一样的：让模型输出一个格式正确的 JSON。但想一想：你要这个 JSON 干什么？在实际工程中，拿到 JSON 之后的用途一般是：查数据库、调用某个 API、在后台系统里创建一条记录、发一封邮件。

function calling（也叫 tool use，工具调用）把这个意图直接摆到了台面上。用退货客服的例子来走一遍。

先从你的代码开始。你在代码里定义一组“工具”。每个工具有三样东西：名字、描述、参数的 JSON Schema。比如你定义了一个工具叫 `create_return_order`，描述是“创建退货工单”，参数 Schema 要求有 `product`（字符串）、`order_id`（字符串）、`reason`（字符串）。

你可能还定义了另一个工具叫 `query_order_status`，描述是“查询订单状态”，参数只需要一个 `order_id`。

你在调用 API 的时候，把这些工具定义和用户的消息一起发给模型。

这一步很关键：模型之所以知道有哪些工具可用，是因为你在这次请求里把工具清单也交给了它。

接着，用户说：“我三天前买的蓝色外套想退货，订单号 20250415123，太大了。”模型看了用户的消息和你定义的工具列表，判断出“这个用户想退货，我应该调用 `create_return_order` 这个工具”。

然后，模型不会直接给用户写一段自然语言回复，而是输出一个结构化的工具调用请求。内容大致长这样：

```
{
  "tool": "create_return_order",
  "arguments": {
    "product": " 蓝色外套",
    "order_id": "20250415123",
    "reason": " 尺码偏大"
  }
}
```

这里的 `arguments` 部分由工具的参数 Schema 严格约束。API 在底层通常会使用前面讲的受限解码或类似机制来保证参数格式正确。

到这里，模型的任务就结束了一半。真正执行动作的是你的代码。你的程序解析出工具名 `create_return_order` 和参数，然后调用你写好的退货工单创建函数，把参数传进去。函数在后台系统里创建了一条退货记录，返回了结果：“退货工单已创建，工单号 RT20250419001”。

有些场景还会多走一步：你把函数的执行结果再发回给模型。模型看到了“退货工单已创建”的结果，再基于这个结果生成一句自然语言回复给用户：“您的退货申请已提交，工单号是 RT20250419001，预计 3 个工作日内处理。”

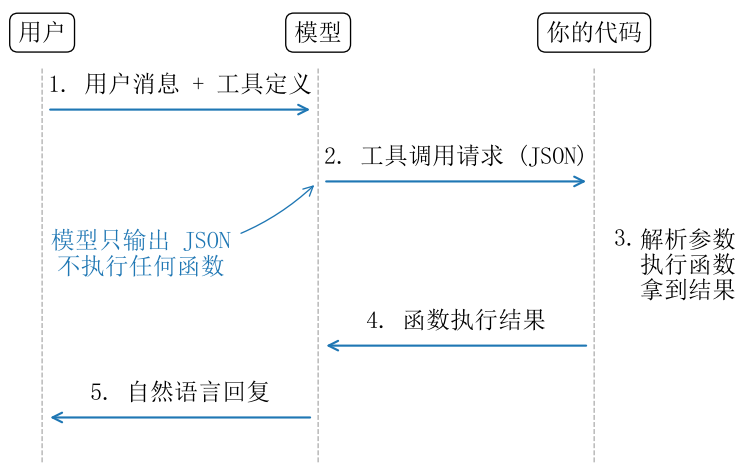


图 13.5: Function calling 的信息流。用户消息和工具定义一起发送给模型（步骤 1）；模型不执行任何函数，只输出一段结构化的工具调用 JSON（步骤 2）；你的代码负责解析参数并调用真实的函数（步骤 3）；函数执行结果发回模型（步骤 4）；模型基于结果生成自然语言回复（步骤 5）。蓝色箭头表示模型参与的交互。

图 13.5 清晰地展示了这条边界：模型的箭头只到你的代码就停了，从不直接通向任何外部系统。模型本身没有调用任何函数。function calling 这个名字容易让人误以为模型自己去执行了什么操作。实际上模型做的事情只有一件：输出一段 JSON，表达“我认为应该调用这个函数，参数是这些”。真正的函数调用发生在你的代码里。模型负责决策，程序负责执行。这个区分在后面接触 Agent 开发的时候会变得非常重要。

从结构化输出的角度看，function calling 其实就是两件事的组合：结构化输出 + 意图路由。结构化输出保证了参数的格式正确（受限解码那一套），意图路由让模型判断应该调用哪个工具（从你定义的工具列表里选）。里面没有一个全新的解码原理。扒

开皮看，你可以把它理解成：结构化输出那套格式约束，加上工具选择这一步。

function calling 的价值不在于发明了什么新技术，而在于它把模型输出结构化数据和数据驱动下游操作天然地绑在了一起。你不需要先让模型输出 JSON，再自己写代码解析 JSON，再根据解析结果决定调哪个函数，function calling 把中间的胶水层省掉了。如果你的场景是模型做决策、程序做执行，function calling 几乎就是首选方案。

不过，function calling 还有一个细节很容易被新人漏掉：模型不一定每次都会调用工具。如果用户问的是“今天天气怎么样”，而你定义的工具只有退货和查单，模型可能判断没有合适的工具可用，直接返回普通文本。这意味着你的代码不能默认返回结果里一定有 `tool_calls`，而是要同时处理两种情况：一种是工具调用请求，另一种是普通文本回复。很多线上小事故就出在这里：代码直接去取 `tool_calls`，没做空值判断，一遇到普通回复就报 `NoneType` 错误。

有些 API 还允许你强制模型必须调用某个工具，这个功能要谨慎用。强制调用意味着即使用户的问题和工具无关，模型也会硬凑一个调用出来，参数很可能是瞎编的。除非业务上真的要求“每次请求都必须落到某个工具”，否则一般让模型自己判断更稳。

13.3.8 最后一道安全网：解析、校验、重试

到这里，你已经看到了几种更硬的方案：受限解码、JSON Mode、JSON Schema、function calling。它们能把格式成功率拉得很高，但线上系统里仍然要留一层安全网。原因很简单：输出可能被截断，API 可能返回空内容，模型可能拒答，网络可能超时，字段的值也可能不符合业务约束。

这层安全网不用写得很玄，核心就是四步：先解析，再校验，失败就重试，最后记录日志。解析负责判断这段文本能不能作为合法 JSON 读取；校验负责判断字段齐不齐、类型对不对、枚举值有没有越界；重试负责把具体错误告诉模型，让它重新输出；日志负责把救不回来的样本留下来，别让一条坏数据拖垮整个批次。

以前那种“找第一个左花括号和最后一个右花括号”的方法，只适合低成本原型。它能救一点模型多说废话的情况，但遇到多个 JSON 对象、字符串里有花括号、输出被截断，就很容易误切。真正上线时，更可靠的做法是优先用 API 提供的 JSON Schema 或 function calling；不支持这些能力时，再把“提取、解析、校验、重试”作为兜底流程。

校验最好交给代码来做。Python 里可以用 Pydantic 这类工具定义字段和类型：缺字段、类型错、枚举值不在范围里，直接报出具体位置。转义、尾逗号、字段类型这些机械活，也尽量交给代码，不要让模型硬背规则。

重试也要有边界。一次失败后，把错误信息带回去让模型重来，通常很有效；连续两三次仍然失败，就不要继续烧钱了。记录原始输入、模型输出、报错信息和使用的 prompt 版本，后面才能分析到底是输入太脏、Schema 太复杂、模型太弱，还是你的格式要求写得不清楚。

13.3.9 拿到一个“要 JSON”的需求，先问三个问题

你在工位上会反复遇到一类需求，表现形式各不相同。有时候是“帮我从这批文本里提取关键字段”，有时候是“模型的回答要能被下游系统解析”，有时候是“我要用模型批量生成训练数据”，但本质上都是同一件事：让模型的输出变成程序可以直接消费的结构化数据。

前面把 prompt 约束、受限解码、JSON Mode、JSON Schema 和 function calling 都拆开讲了一遍。真正坐到工位上接需求的时候，你不需要把这些方案重新背一遍。每次拿到这种需求，先问三个问题，答案会把你直接导向最合适的方案。

第一个问题：你用的是什么模型，怎么部署的？

如果你用的是主流闭源 API（比如通过 API 调用大模型），先去翻一下文档，看它支持到哪一层：JSON Mode、JSON Schema 模式，还是 function calling。只支持 JSON Mode，至少能保证输出是合法 JSON；支持 JSON Schema 模式，结构就能被卡住；支持 function calling，模型还能顺手帮你做工具选择。先把平台能给你的硬约束摸清楚，再往下判断需求。

如果你用的是本地部署的开源模型，看一下你的推理框架（比如 vLLM）有没有 guided decoding（也就是前面讲的受限解码）功能。有的话，效果和闭源 API 的 JSON Schema 模式基本等价。没有的话，你的起点只能是“prompt 约束 + 后处理”。别灰心，很多线上项目就是这么跑的。

第二个问题：你的输出结构是固定的，还是动态变化的？

如果每条数据的输出结构都一样，也就是同样的字段名、同样的类型、同样的嵌套，这是最适合 JSON Schema 模式的场景。定义一次 Schema，所有数据复用。

如果你的 API 只支持 JSON Mode，没有 JSON Schema 模式，也不是不能用。它至少能帮你挡掉尾逗号、少括号、多废话这些语法问题。只是字段缺失、字段名漂移、类型不对这些结构问题，仍然要靠后处理校验来兜底。

如果输出结构会根据输入变化，比如不同类型的商品需要提取不同的字段，你就需要在代码里维护多套 Schema 定义和对应的解析逻辑，工程复杂度会上升。这种情况下，prompt 约束 + 后处理反而更灵活。你可以在 prompt 里根据输入类型动态拼接不同的格式要求和示例。

第三个问题：你的需求是“得到一个 JSON”，还是“让模型驱动下游操作”？

如果你只是需要把模型的输出解析成结构化数据存起来、做分析、或者喂给另一个模型，优先考虑 JSON Schema 模式。如果平台只支持 JSON Mode，就用 JSON Mode 配后处理校验；再退一步，才是 prompt 约束 + 后处理。

如果你需要模型根据用户的请求，判断应该执行什么操作、传什么参数，比如用户说“帮我查一下订单状态”，模型要判断“应该调用查询订单的函数，参数是订单号”，这是 function calling 的典型场景。它比单纯的结构化输出多了一层“意图路由”的能力。

三个问题回答完，你的方案基本就定了。但不管选了哪个方案，有一件事永远要做：**在后处理中加一层校验**。哪怕你用了 JSON Schema 模式、理论上格式不可能出错，也在代码里加一个 `try-except` 包住 `json.loads()`，加一个 Pydantic 模型做字段校验。这么做的理由很朴素：线上系统需要对所有异常情况都有兜底，网络超时、API 返回格式变化、模型版本静默更新，这些事随时可能发生。防御性编程的成本极低，但它能在关键时刻救你一命。

还有一条贯穿所有方案的原则：**先试成本最低的，不行再升级**。如果你的场景是每天跑几十条、容忍偶尔出错，prompt 约束就够了。如果 prompt 约束的成功率不够、或者你的量级上来了，先看平台有没有 JSON Mode；JSON Mode 够用就先用它，再配后处理校验。还不够，或者你需要确定性的字段结构，再升到 JSON Schema 模式。需要让模型决定调用哪个工具，再用 function calling。不要一上来就用最重的方案。工程中的好决策，关键是用刚好够用的工具。

13.3.10 实战中几个隐蔽的坑

最后聊几个教科书里很少写、但实际项目里很容易碰到的坑。

- **结构化输出本身也会吃 token。**

字段名、冒号、逗号、双引号、示例里的完整 JSON 骨架，都会被算进输入或输出成本里。字段很多、调用量很大时，字段名尽量简短，极少出现的字段可以考虑设为可选，不要让模型每次都输出一堆 `null`。

- **温度参数和结构化输出的关系。**

前面讲解码策略的时候说过，温度越高，模型输出越随机、越有创造力。但在结构化输出的场景里，创造力恰恰是你不想要的。你希望模型老老实实地按格式输出，不要搞创新。所以做结构化提取的时候，通常把 `temperature` 设得很低，接近 0。如果你用了受限解码或 JSON Schema 模式，温度的影响会小很多（因为不合法的 token 已经被屏蔽了），但如果你只靠 prompt 约束，温度拧高是在给自己找麻烦。

- **Schema 太复杂，模型会分心。**

当你的 JSON 结构有三四层嵌套、十几个字段、每个字段还有各种条件约束的时候，你会发现一个奇怪的现象：格式倒是对了，但字段里填的内容质量明显下降。模型并没有变笨，它只是把太多注意力分给了“搞清楚该输出什么结构”，留给认真理解输入内容的注意力就少了。实战中的应对策略是：把一个复杂的提取任务拆成多次调用，每次只提取一部分信息，每次的 Schema 保持简单。

- **截断是最常被忽视的隐患。**

前面在翻车案例里提过，这里再强调一次防御方式：根据你的 Schema 估算最长可能的输出长度，给 `max_tokens` 留出足够余量。在后处理中加一个简单的检查：如果输出的最后一个非空字符不是 `}` 或 `]`，大概率是被截断了，直接走重试，不要浪费时间去修复一个残缺的 JSON。

- **小模型的格式遵从能力弱得多。**

如果你只用过大的闭源模型，可能对这个问题完全没有感知。但当你尝试在 7B 甚至更小的开源模型上用 prompt 约束来获取 JSON 输出时，你会发现成功率断崖式下降。同样的 prompt，在大模型上 90% 的成功率，在小模型上可能只有 50%。这通常不是 prompt 写得差，问题更多出在小模型对指令的遵从能力本身就弱。它的参数量不够支撑它同时理解输入内容和遵守格式约束。在本地部署小模型的场景里，受限解码经常是上线门槛。

- **流式输出和结构化输出有冲突。**

很多 API 默认是流式返回的。模型生成一个 token 就发一个 token 回来，让用户看到“正在打字”的效果。但流式返回意味着你在收到完整输出之前没法做 JSON 解析。如果你的场景需要结构化输出，要么关掉流式、等模型全部生成完再一次性拿到结果去解析；要么在流式接收的过程中先把 token 拼起来，前端可以展示“正在生成”的状态，让用户知道系统没卡住，但 JSON 解析要等全部收完之后再做。这不是什么大坑，但第一次碰到的时候容易困惑。

- **字段名用中文还是英文。**

一句话的建议：字段名永远用英文，值里面该中文就中文。字段名是给程序读的，英文不会有编码问题，也和绝大多数编程语言的命名习惯一致。值是给业务用的，中文数据就用中文，不需要翻译。

13.3.11 把两个问题分清楚

这一节讲了很多方案和技巧，但在结束之前，有一个贯穿始终的认知值得单独拎出来强调，它能帮你在面对结构化输出问题时少走弯路：

格式正确和内容正确是两个完全独立的问题。

受限解码、JSON Schema、function calling，这些方案解决的全部是格式问题。它们保证输出的 JSON 语法合法、结构符合预期。但 JSON 里填的内容对不对，提取的名称是不是原文里的、数字是不是准确的、分类是不是正确的，这些，它们一概管不了。

打个比方：这些方案保证的是信封的形状和尺寸完全符合邮局的规格要求。但信封里装的那封信写了什么、写得对不对，和信封的形状没有半点关系。

内容质量取决于模型本身的能力、prompt 的质量、以及输入数据的清晰程度。这些是前面 prompt engineering 讲的东西，以及后面微调相关章节要解决的问题。

把这两件事分清楚，你在排查问题的时候就不会走弯路。下游系统报错了，先判断是格式问题还是内容问题。格式问题用这一节讲的方案解决。内容问题回去优化 prompt、改进数据、或者换一个更强的模型。两类问题的解法完全不同，混在一起只会越修越乱。

13.3.12 自检清单

1. prompt 约束为什么不能保证模型每次都输出合法 JSON？

因为模型是逐 token 生成文本的，prompt 里的格式要求只是众多上下文信号之一。它能提高正确格式出现的概率，但无法保证每一步都选出语法正确的 token。一旦某一步生成偏了，后续的错误会被放大而不是被纠正。

2. JSON Mode 和 JSON Schema 模式有什么区别？

JSON Mode 只保证输出是语法合法的 JSON，不保证 JSON 的具体结构。JSON Schema 模式保证输出不仅语法合法，而且严格符合你传入的 Schema 定义。前者管语法，后者管语法加结构。

3. 受限解码的核心思路是什么？

在模型生成每个 token 之前，根据 JSON 语法规则判断哪些 token 在当前位置是合法的，把不合法 token 的 logit 设为负无穷（Softmax 之后概率为零），强制模型只能从合法 token 中选择。

4. 受限解码能保证内容正确吗？

不能。它只保证语法正确，不保证语义正确。格式正确和内容正确是两个独立的问题。

5. Function calling 中，是谁在调用函数？

不是模型。模型只是输出了一段 JSON，表达“我认为应该调用这个函数，参数是这些”。真正的函数调用发生在你的代码里。模型是决策者，不是执行者。

13.3.13 Python 交互实验室

这一节讲了六种翻车姿势、源头约束、兜底流程和方案选型，信息量不小。配套的 `structured_json_demo.py` 准备了 8 种典型的翻车样本，会展示 `json.loads()` 的报错信息、尾逗号修复前后的解析结果变化、字段名不一致时校验步骤如何定位问题，让你把“提取 → 解析 → 校验 → 生成重试 prompt”的完整兜底流程走一遍。跑完之后，最值得做的一件事是：把你自己在项目中遇到过的（或者能想象到的）翻车输出，添加

到代码开头的样本列表里，看看这层安全网能不能接住。还可以试试往字段校验规则里加一个 `float` 类型的价格字段，观察校验步骤的反应，这会帮你理解语法正确和结构正确之间的差距。